KiteLang - Compiler Construction Project

Linux build and usage guide for the Phase 1 interpreter and Phase 2 LLVM AOT compiler.

Dependencies

Install **build-essential**, **flex**, **bison**, **clang**, and **llvm**. On Ubuntu/Debian:

```
sudo apt update
sudo apt install -y build-essential flex bison clang llvm
```

Build

```
make
```

This creates build/kitelang-interpreter and build/kitelang-compiler. Use `make clean` to remove generated files.

Phase 1: Interpreter

The Bison grammar constructs and directly executes the program AST; no intermediate code is created.

```
./build/kitelang-interpreter examples/insertion_sort.kite
```

Phase 2: AOT compiler

The compiler emits textual LLVM IR. Clang turns that IR into a native Linux executable.

```
./build/kitelang-compiler examples/insertion_sort.kite -o build/program.ll
clang build/program.ll -o build/program
./build/program
```

Language use

```
int data[3] = {9, 4, 1};
pecha (data[0] > data[1]) { palta(data[0], data[1]); }
dor (i : 0 -> 2) { print(data[i]); }
```

KiteLang uses pecha/warna for if/else, dheel for while, dor for an inclusive increasing range loop, and palta to exchange two variables or array elements. The implemented runtime core uses integers; float and bool declaration keywords are recognized and share that representation.

Project files

src/kitelang.l: Flex lexer. src/kitelang.y: Bison grammar, AST, interpreter, and LLVM emitter. src/ast.hpp: shared AST declarations. examples/insertion_sort.kite: sample program. Makefile: Linux build commands. KiteLang_Proposal.pdf: Phase 0 proposal.